

1

```
class str {  
    public static void main (String args[]) {  
        for (int i = 1; i <= 10; i++) {  
            System.out.println("Hello world");  
        }  
    }  
}
```

1b

```
import java.util.*;  
public class main {  
    public static void main (String arg[]) {  
        int a = 10, b = 20, c = 10, d = 5;  
        int result = a + b * c % d;  
  
        System.out.println("Result is: " + result);  
    }  
}
```

2a

```
import java.util.*;  
public class main {  
    public static void main (String args[]) {  
        int a = 23, b = 45, c = 12;  
        if (a > b && a > c) {  
            System.out.println("A is greater");  
        } else if (b > a && b > c) {  
            System.out.println("B is greater");  
        } else {  
            System.out.println("C is greater");  
        }  
    }  
}
```

```
}
```

2b

```
import java.util.*;
```

```
class marks {
```

```
    public static void main (String args[]) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        System.out.println("Enter your marks");
```

```
        int m = sc.nextInt();
```

```
        switch (m / 10) {
```

```
            case 10:
```

```
                System.out.println("A+ grade");
```

```
                break;
```

```
            case 9:
```

```
                System.out.println("A grade");
```

```
                break;
```

```
            case 8:
```

```
                System.out.println("B+ grade");
```

```
                break;
```

```
            case 7:
```

```
                System.out.println("B grade");
```

```
                break;
```

```
            case 6:
```

```
                System.out.println("C+ grade");
```

```
                break;
```

```
            case 5:
```

```
                System.out.println("C grade");
```

```
                break;
```

```
            default:
```

```
                System.out.println("D grade");
```

```
        }
```

```
        sc.close();
```

```
    }  
}
```

3a

```
import java.util.*;  
  
class circle {  
    double radius;  
    double area() {  
        return 3.14 * radius * radius;  
    }  
}  
  
class oops {  
    public static void main(String args[]) {  
        Scanner sc = new Scanner(System.in);  
        circle obj = new circle();  
        System.out.println("Enter radius");  
        obj.radius = sc.nextDouble();  
        double calculate = obj.area();  
        System.out.println("Area of circle is: " + calculate);  
        sc.close();  
    }  
}  
  
import java.util.*;  
  
class rectangle {  
    int length, breadth;  
    void setdata(int l, int b) {  
        length = l;  
        breadth = b;  
    }  
    int calculate() {  
        return length * breadth;  
    }  
}
```

```
}  
}
```

```
class area extends rectangle {  
    public static void main(String args[]) {  
        area aa = new area();  
        aa.setdata(5, 4);  
        System.out.println("Area of rectangle is: " + aa.calculate());  
    }  
}
```

3c

```
import java.util.*;  
class emp1 {  
    int id;  
    Scanner sc = new Scanner(System.in);  
    void input1() {  
        System.out.println("Enter id: ");  
        id = sc.nextInt();  
    }  
}
```

```
class emp2 extends emp1 {  
    int age;  
    Scanner sc = new Scanner(System.in);  
    void input2() {  
        System.out.println("Enter Age: ");  
        age = sc.nextInt();  
    }  
}
```

```
class emp3 extends emp2 {
```

```

String name;

Scanner sc = new Scanner(System.in);

void input3() {
    System.out.println("Enter name: ");
    name = sc.nextLine();
}
}

class employee {
    public static void main(String args[]) {
        System.out.println("\nZahid Yakheen 33 SYIT");
        emp3 obj = new emp3();
        obj.input1();
        obj.input2();
        obj.input3();
        obj.display(); // Note: display() method is not shown in the PDF snippet
    }
}

```

3d

```

import java.util.*;

class emp1 {
    int id;
    String name;
    void input1() {
        Scanner sc = new Scanner(System.in);
        System.out.println("enter id: ");
        id = sc.nextInt();
        sc.nextLine();
        System.out.println("enter name: ");
        name = sc.nextLine();
    }
}

```

```
}
```

```
class fulltime extends emp1 {  
    double salary;  
    String department;  
    void input2() {  
        Scanner sc = new Scanner(System.in);  
        System.out.println("enter salary: ");  
        salary = sc.nextDouble();  
        sc.nextLine();  
        System.out.println("enter department: ");  
        department = sc.nextLine();  
    }  
    void fulltimedetails() {  
        System.out.println("--FULLTIME EMPLOYEE--");  
        System.out.println("Name: " + name);  
        System.out.println("ID: " + id);  
        System.out.println("Salary: " + salary);  
        System.out.println("Department: " + department);  
    }  
}
```

```
class parttime extends emp1 {  
    int hours;  
    double rate;  
    void input3() {  
        Scanner sc = new Scanner(System.in);  
        System.out.println("Enter rate per hour: ");  
        rate = sc.nextDouble();  
        System.out.println("Enter working hours: ");  
        hours = sc.nextInt();  
    }  
}
```

```
}
```

```
void parttimeDetails() {  
    System.out.println("--PART TIME EMPLOYEE DETAILS--");  
    System.out.println("Name: " + name);  
    System.out.println("ID: " + id);  
    System.out.println("Rate per hour: " + rate);  
    System.out.println("Working hours: " + hours);  
    System.out.println("Total Salary: " + (rate * hours));  
}  
}
```

```
class partemp {  
    public static void main(String args[]) {  
        int choice;  
        Scanner sc = new Scanner(System.in);  
        System.out.println("1. Full time Employee");  
        System.out.println("2. Part time Employee");  
        System.out.println("Enter choice: ");  
        choice = sc.nextInt();  
  
        if (choice == 1) {  
            fulltime fobj = new fulltime();  
            fobj.input1();  
            fobj.input2();  
            fobj.fulltimedetails();  
        } else if (choice == 2) {  
            parttime pobj = new parttime();  
            pobj.input1();  
            pobj.input3();  
            pobj.parttimeDetails();  
        }  
    }  
}
```

```

        } else {
            System.out.println("Invalid choice");
        }
    }
}

```

4

```

import java.util.Scanner;

class shape {
    void displaytype() {
        System.out.println("this is shape");
    }
}

class circle extends shape {
    double radius;
    void getradius() {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter radius: ");
        radius = sc.nextDouble();
    }
    void area() {
        double area = 3.14 * radius * radius;
        System.out.println("Area of circle = " + area);
    }
}

```

```

class rectangle extends shape {
    double length, breadth;
    void getdimensions() {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter length: ");
    }
}

```



```

length = sc.nextDouble();

System.out.println("Enter breadth: ");

breadth = sc.nextDouble();
}

void area() {
    double area = length * breadth;

    System.out.println("Area of rectangle = " + area);
}
}

class Main {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);

        System.out.println("1. Circle");
        System.out.println("2. Rectangle");
        System.out.println("Enter your choice: ");

        int ch = sc.nextInt();

        if (ch == 1) {
            circle c = new circle();

            c.displaytype();

            c.getradius();

            c.area();
        } else if (ch == 2) {
            rectangle r = new rectangle();

            r.displaytype();

            r.getdimensions();

            r.area();
        } else {
            System.out.println("Invalid choice");
        }
    }
}

```

```

    }
}
4
import java.util.Scanner;
class shape {
    void displaytype() {
        System.out.println("this is shape");
    }
}

class circle extends shape {
    double radius;
    void getradius() {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter radius: ");
        radius = sc.nextDouble();
    }
    void area() {
        double area = 3.14 * radius * radius;
        System.out.println("Area of circle = " + area);
    }
}

class rectangle extends shape {
    double length, breadth;
    void getdimensions() {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter length: ");
        length = sc.nextDouble();
        System.out.println("Enter breadth: ");
        breadth = sc.nextDouble();
    }
}

```

```

    }

    void area() {
        double area = length * breadth;
        System.out.println("Area of rectangle = " + area);
    }
}

```

```

class Main {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        System.out.println("1. Circle");
        System.out.println("2. Rectangle");
        System.out.println("Enter your choice: ");
        int ch = sc.nextInt();
        if (ch == 1) {
            circle c = new circle();
            c.displaytype();
            c.getradius();
            c.area();
        } else if (ch == 2) {
            rectangle r = new rectangle();
            r.displaytype();
            r.getdimensions();
            r.area();
        } else {
            System.out.println("Invalid choice");
        }
    }
}

import java.util.*;

```

```
class Department {  
    String deptName;  
    Department(String name) {  
        this.deptName = name;  
    }  
    void displayInfo() {  
        System.out.println("Department Name: " + deptName);  
    }  
    void subjects() {  
        System.out.println("Common subjects for all departments.");  
    }  
}
```

```
class ITDepartment extends Department {  
    ITDepartment() {  
        super("Information Technology");  
    }  
    void subjects() {  
        System.out.println("IT subjects: Java, Networking, DBMS");  
    }  
}
```

```
class CSDepartment extends Department {  
    CSDepartment() {  
        super("Computer Science");  
    }  
    void subjects() {  
        System.out.println("CS subjects: Python, Data Structures, OS");  
    }  
}
```

```

class AIMLDepartment extends Department {
    AIMLDepartment() {
        super("Artificial Intelligence and Machine Learning");
    }
    void subjects() {
        System.out.println("AI-ML subjects: Machine Learning, AI, Deep Learning");
    }
}

```

```

public class College {
    public static void main(String args[]) {
        ITDepartment it = new ITDepartment();
        CSDepartment cs = new CSDepartment();
        AIMLDepartment aiml = new AIMLDepartment();

        System.out.println("--IT Department--");
        it.displayInfo();
        it.subjects();

        System.out.println("\n--CS Department--");
        cs.displayInfo();
        cs.subjects();

        System.out.println("\n--AIML Department--");
        aiml.displayInfo();
        aiml.subjects();
    }
}

```

4

```

import java.util.*;
class employee {

```

```
String name;
int id;
void setdata(String n, int i) {
    name = n;
    id = i;
}
void showData() {
    System.out.println("Name: " + name);
    System.out.println("ID: " + id);
}
}
```

```
class teacher extends employee {
    void subject() {
        System.out.println("Subject: Computer Science");
    }
}
```

```
class clerk extends employee {
    void worktype() {
        System.out.println("Work: Office Administration");
    }
}
```

```
public class college {
    public static void main(String args[]) {
        teacher t = new teacher();
        clerk c = new clerk();
        t.setdata("Rohit", 101);
        c.setdata("Saud", 202);
        System.out.println("--TEACHER DETAILS--");
    }
}
```

```

        t.showData();

        t.subject();

        System.out.println("\n--CLERK DETAILS--");

        c.showData();

        c.worktype();

    }

}

```

5a

```

class Product {

    String productName;

    double price;

    void setProduct(String name, double p) {

        productName = name;

        price = p;

    }

    void show() {

        System.out.println("Product: " + productName);

        System.out.println("Price: " + price);

    }

}

```

```

class Electronics extends Product {

    void warranty() {

        System.out.println("Warranty: 2 years");

    }

}

```

```

class Clothes extends Product {

    void material() {

        System.out.println("Material: Cotton");

    }

}

```

```
}
```

```
class Groceries extends Product {  
    void expiry() {  
        System.out.println("Expiry: 6 months");  
    }  
}
```

```
public class Shopping {  
    public static void main(String args[]) {  
        Electronics e = new Electronics();  
        Clothes c = new Clothes();  
        Groceries g = new Groceries();  
  
        e.setProduct("Laptop", 55000);  
        c.setProduct("T-shirt", 799);  
        g.setProduct("Rice", 1200);  
  
        e.show(); e.warranty();  
        c.show(); c.material();  
        g.show(); g.expiry();  
    }  
}
```

5b

```
class Calculator {  
    int n1, n2, n3;  
    Calculator(int a, int b, int c) {  
        n1 = a;  
        n2 = b;  
        n3 = c;  
    }  
}
```



```

int getSum() {
    return n1 + n2 + n3;
}
}

```

```

class AverageCalculator extends Calculator {
    AverageCalculator(int a, int b, int c) {
        super(a, b, c);
    }
    double getAverage() {
        int sum = super.getSum();
        return sum / 3.0;
    }
}

```

```

public class main {
    public static void main(String[] args) {
        AverageCalculator obj = new AverageCalculator(10, 20, 30);
        int sum = obj.getSum();
        double avg = obj.getAverage();
        System.out.println("Sum = " + sum);
        System.out.println("Average = " + avg);
    }
}

```

5c

```

class AverageCalculator {
    double average(int a, int b, int c) {
        return (a + b + c) / 3.0;
    }
    double average(double a, double b, double c) {
        return (a + b + c) / 3;
    }
}

```

```
}  
}
```

```
public class main {  
    public static void main(String[] args) {  
        AverageCalculator obj = new AverageCalculator();  
        double avg1 = obj.average(10, 20, 30);  
        double avg2 = obj.average(10.5, 20.5, 30.5);  
  
        System.out.println("Average of integers: " + avg1);  
        System.out.println("Average of doubles: " + avg2);  
    }  
}
```

5d

```
class ElectricityBill {  
    double calculateBill(int units) {  
        double rate = 5.50;  
        return units * rate;  
    }  
  
    double calculateBill(int units, double commercialRate) {  
        return units * commercialRate;  
    }  
  
    double calculateBill(int units, double industrialRate, double demandCharge) {  
        return (units * industrialRate) + demandCharge;  
    }  
}
```

```
public class main {  
    public static void main(String[] args) {  
        ElectricityBill bill = new ElectricityBill();  
        double domestic = bill.calculateBill(130);  
    }  
}
```

```

double commercial = bill.calculateBill(200, 8.75);
double industrial = bill.calculateBill(500, 10.25, 1500);

System.out.println("Domestic Bill: ₹ " + domestic);
System.out.println("Commercial Bill: ₹ " + commercial);
System.out.println("Industrial Bill: ₹ " + industrial);
}
}
6a
// Experiment 6a: Area and perimeter of a circle using abstract class
abstract class Shape {
    abstract void findArea();
    abstract void findPerimeter();
}

class Circle extends Shape {
    double radius;

    Circle(double radius) {
        this.radius = radius;
    }

    void findArea() {
        double area = Math.PI * radius * radius;
        System.out.println("Area of Circle: " + area);
    }

    void findPerimeter() {
        double perimeter = 2 * Math.PI * radius;
        System.out.println("Perimeter (circumference): " + perimeter);
    }
}

```

```
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Circle c = new Circle(5);  
        c.findArea();  
        c.findPerimeter();  
    }  
}
```

6b

// Experiment 6b: Runtime polymorphism using abstract classes

```
abstract class Payment {  
    abstract void pay(int amount);  
  
    void welcome() {  
        System.out.println("Welcome to Payment Portal");  
    }  
}  
  
class CreditCard extends Payment {  
    void pay(int amount) {  
        System.out.println("Paid " + amount + " using Credit Card");  
    }  
}  
  
class UPI extends Payment {  
    void pay(int amount) {  
        System.out.println("Paid " + amount + " using UPI");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Payment p1 = new CreditCard();  
        Payment p2 = new UPI();  
  
        p1.welcome();  
        p1.pay(1500);  
        p2.pay(500);  
    }  
}
```

7a

// Experiment 7a: Constructor overloading using Student class

```
class Student {  
    int rollNo;  
    String name;  
  
    // Constructor with no parameters  
    Student() {  
        rollNo = 0;  
        name = "Not Assigned";  
    }  
  
    // Constructor with one parameter  
    Student(int r) {  
        rollNo = r;  
        name = "Not Assigned";  
    }  
  
    // Constructor with two parameters  
    Student(int r, String n) {  
        rollNo = r;
```

```

        name = n;
    }

    void display() {
        System.out.println(rollNo + " " + name);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student();
        Student s2 = new Student(101);
        Student s3 = new Student(102, "Rahul");

        s1.display();
        s2.display();
        s3.display();
    }
}

```

7b

// Experiment 7b: Constructor overloading for BankAccount

```

class BankAccount {
    int accNo;
    String name;
    double balance;

    // Constructor 1: only account number
    BankAccount(int a) {
        accNo = a;
        name = "Not Provided";
        balance = 0.0;
    }
}

```

```
}
```

```
// Constructor 2: account number and name
```

```
BankAccount(int a, String n) {
```

```
    accNo = a;
```

```
    name = n;
```

```
    balance = 0.0;
```

```
}
```

```
// Constructor 3: account number, name, balance
```

```
BankAccount(int a, String n, double b) {
```

```
    accNo = a;
```

```
    name = n;
```

```
    balance = b;
```

```
}
```

```
void display() {
```

```
    System.out.println("Account No: " + accNo);
```

```
    System.out.println("Name: " + name);
```

```
    System.out.println("Balance: " + balance);
```

```
    System.out.println("-----");
```

```
}
```

```
}
```

```
public class ConstructorOverloadingBank {
```

```
    public static void main(String[] args) {
```

```
        BankAccount b1 = new BankAccount(101);
```

```
        BankAccount b2 = new BankAccount(102, "Ajay");
```

```
        BankAccount b3 = new BankAccount(103, "Neha", 5000);
```

```
        b1.display();
```

```
        b2.display();  
        b3.display();  
    }  
}
```

7c

// Experiment 7c: Constructor overloading using Mobile class

```
class Mobile {  
    String brand;  
    int price;  
    int storage;  
  
    // Constructor 1  
    Mobile(String b) {  
        brand = b;  
        price = 0;  
        storage = 0;  
    }  
  
    // Constructor 2  
    Mobile(String b, int p) {  
        brand = b;  
        price = p;  
        storage = 64;  
    }  
  
    // Constructor 3  
    Mobile(String b, int p, int s) {  
        brand = b;  
        price = p;  
        storage = s;  
    }  
}
```



```
void details() {  
    System.out.println(brand + " " + price + " " + storage + "GB");  
}  
}
```

```
public class MobileDemo {  
    public static void main(String[] args) {  
        Mobile m1 = new Mobile("Samsung");  
        Mobile m2 = new Mobile("Redmi", 12000);  
        Mobile m3 = new Mobile("Apple", 70000, 256);  
  
        m1.details();  
        m2.details();  
        m3.details();  
    }  
}
```

8a

// Interface example for different vehicles

```
interface Vehicle {  
    void start();  
}
```

```
class Car implements Vehicle {  
    public void start() {  
        System.out.println("Car starts with key");  
    }  
}
```

```
class Bike implements Vehicle {  
    public void start() {
```

```
        System.out.println("Bike starts with self-start");
    }
}
```

```
public class VehicleDemo {
    public static void main(String[] args) {
        Vehicle v;

        v = new Car();
        v.start();

        v = new Bike();
        v.start();
    }
}
```

8b

// Multiple inheritance style using interfaces

```
interface Employee {
    void work();
}
```

```
class Developer implements Employee {
    public void work() {
        System.out.println("Developer writes and maintains code");
    }
}
```

```
class Tester implements Employee {
    public void work() {
        System.out.println("Tester tests the application for bugs");
    }
}
```

```
}
```

```
public class EmployeeDemo {  
    public static void main(String[] args) {  
        Employee e;  
  
        e = new Developer();  
        e.work();  
  
        e = new Tester();  
        e.work();  
    }  
}
```

8c

// Experiment 8.c: Multiple interfaces in one program (real life)

```
interface Employee {  
    void work();  
    double calculateSalary();  
}
```

// Interface for bonus

```
interface Bonus {  
    double calculateBonus();  
}
```

```
class Developer implements Employee, Bonus {  
    public void work() {  
        System.out.println("Developer writes code");  
    }
```

```
    public double calculateSalary() {
```

```

        return 50000;
    }

    public double calculateBonus() {
        return 10000;
    }
}

class Tester implements Employee, Bonus {
    public void work() {
        System.out.println("Tester tests the application");
    }

    public double calculateSalary() {
        return 40000;
    }

    public double calculateBonus() {
        return 8000;
    }
}

public class EmployeeDemo {
    public static void main(String[] args) {
        Employee e1 = new Developer();
        Employee e2 = new Tester();

        System.out.println("Developer Details:");
        e1.work();
        System.out.println("Salary: " + e1.calculateSalary());
        System.out.println("Bonus: " + ((Bonus) e1).calculateBonus());
    }
}

```

```

        System.out.println("-----");

        System.out.println("Tester Details:");

        e2.work();

        System.out.println("Salary: " + e2.calculateSalary());

        System.out.println("Bonus: " + ((Bonus) e2).calculateBonus());

        System.out.println("-----");
    }
}

```

8d

// Experiment 8.d: Multiple interfaces in one program

// ----- INTERFACE 1 -----

```

interface Vehicle {
    void start();
    void stop();
}

```

// ----- INTERFACE 2 -----

```

interface User {
    void login();
    void logout();
}

```

// ----- INTERFACE 3 -----

```

interface Payment {
    void pay(double amount);
}

```

// ----- INTERFACE 4 -----

```

interface Academic {

```

```
void uploadData();
}

// ----- VEHICLE IMPLEMENTATION -----
class Car implements Vehicle {
    public void start() {
        System.out.println("Car starts with key");
    }

    public void stop() {
        System.out.println("Car stops using brakes");
    }
}

// ----- USER + ACADEMIC IMPLEMENTATION -----
class Student implements User, Academic {
    public void login() {
        System.out.println("Student logged in");
    }

    public void logout() {
        System.out.println("Student logged out");
    }

    public void uploadData() {
        System.out.println("Student uploaded assignment");
    }
}

// ----- PAYMENT IMPLEMENTATION -----
class UPIPayment implements Payment {
```

```

    public void pay(double amount) {
        System.out.println("Paid " + amount + " using UPI");
    }
}

// ----- MAIN CLASS -----

public class AllInOneInterfaceProgram {
    public static void main(String[] args) {

        System.out.println("Vehicle Interface Example");
        Vehicle v = new Car();
        v.start();
        v.stop();
        System.out.println("-----");

        System.out.println("User & Academic Interface Example");
        User u = new Student();
        u.login();
        ((Academic) u).uploadData();
        u.logout();
        System.out.println("-----");

        System.out.println("Payment Interface Example");
        Payment p = new UPIPayment();
        p.pay(1500);
    }
}

9a

// Method overriding: Display interest rate of different banks
class Bank {
    double getInterestRate() {

```

```
        return 5.0;
    }
}
```

```
class SBI extends Bank {
    double getInterestRate() {
        return 6.5;
    }
}
```

```
class HDFC extends Bank {
    double getInterestRate() {
        return 7.0;
    }
}
```

```
class ICICI extends Bank {
    double getInterestRate() {
        return 6.8;
    }
}
```

```
public class MethodOverridingDemo {
    public static void main(String[] args) {
        Bank bank;

        bank = new SBI();
        System.out.println("SBI Interest Rate: " + bank.getInterestRate());

        bank = new HDFC();
        System.out.println("HDFC Interest Rate: " + bank.getInterestRate());
    }
}
```



```
        bank = new ICICI();  
        System.out.println("ICICI Interest Rate: " + bank.getInterestRate());  
    }  
}
```

9b

// Abstract class BankAccount with Taxable interface

```
interface Taxable {  
    double calculateTax();  
}
```

```
abstract class BankAccount {  
    protected String accountHolder;  
    protected double balance;
```

```
    BankAccount(String accountHolder, double balance) {  
        this.accountHolder = accountHolder;  
        this.balance = balance;  
    }
```

```
    abstract void calculateInterest();
```

```
    final void displayAccount() {  
        System.out.println("Account Holder: " + accountHolder);  
        System.out.println("Balance: " + balance);  
    }  
}
```

```
class SavingsAccount extends BankAccount implements Taxable {
```

```
    SavingsAccount(String name, double balance) {  
        super(name, balance);
```

```
}
```

```
@Override
```

```
void calculateInterest() {
```

```
    double interest = balance * 0.04;
```

```
    balance += interest;
```

```
    System.out.println("Savings Interest Added: " + interest);
```

```
}
```

```
@Override
```

```
public double calculateTax() {
```

```
    return balance * 0.02;
```

```
}
```

```
}
```

```
class CurrentAccount extends BankAccount {
```

```
    CurrentAccount(String name, double balance) {
```

```
        super(name, balance);
```

```
}
```

```
@Override
```

```
void calculateInterest() {
```

```
    System.out.println("No interest for Current Account");
```

```
}
```

```
}
```

```
public class BankInheritanceDemo {
```

```
    public static void main(String[] args) {
```

```
        BankAccount acc1 = new SavingsAccount("Rahul", 5000.0);
```

```
        BankAccount acc2 = new CurrentAccount("Rahul", 1000.0);
```

```
acc1.calculateInterest();
```

```
acc1.displayAccount();
```

```
System.out.println();
```

```
acc2.calculateInterest();
```

```
acc2.displayAccount();
```

```
System.out.println();
```

```
Taxable t = new SavingsAccount("Amit", 6000.0);
```

```
System.out.println("Tax to be paid: " + t.calculateTax());
```

```
}
```

```
}
```

```
9c
```

```
// Abstract Employee with Salary interface
```

```
interface Salary {
```

```
    void calculateSalary();
```

```
}
```

```
abstract class Employee {
```

```
    String name;
```

```
    int empId;
```

```
    Employee(String name, int empId) {
```

```
        this.name = name;
```

```
        this.empId = empId;
```

```
    }
```

```
    void showDetails() {
```

```
        System.out.println("Employee Name: " + name);
```

```
        System.out.println("Employee ID: " + empId);
```

```
}
```

```
abstract void work();
```

```
}
```

```
class Developer extends Employee implements Salary {
```

```
    Developer(String name, int empld) {
```

```
        super(name, empld);
```

```
    }
```

```
@Override
```

```
void work() {
```

```
    System.out.println("Developer writes and tests code.");
```

```
}
```

```
@Override
```

```
public void calculateSalary() {
```

```
    System.out.println("Salary calculated based on projects and experience.");
```

```
}
```

```
}
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Developer dev = new Developer("Anik", 101);
```

```
        dev.showDetails();
```

```
        dev.work();
```

```
        dev.calculateSalary();
```

```
    }
```

```
}
```

10a

```
// Sort characters of a string in ascending order
```

```

import java.util.Arrays;
import java.util.Scanner;

public class SortStringAscending {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a string: ");
        String str = sc.nextLine();

        // Convert string to character array
        char[] charArray = str.toCharArray();

        // Sort the character array
        Arrays.sort(charArray);

        // Convert back to string
        String sortedString = new String(charArray);

        // Output
        System.out.println("String in ascending order: " + sortedString);
    }
}

```

10b

// Sort characters of a string in ascending and descending order

```

import java.util.Arrays;
import java.util.Scanner;

public class SortStringAscendingDescending {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

```

```

System.out.print("Enter a string: ");

String str = sc.nextLine();


// Convert string to character array
char[] charArray = str.toCharArray();


// Sort in ascending order
Arrays.sort(charArray);

String ascending = new String(charArray);

System.out.println("String in Ascending order: " + ascending);


// Reverse for descending
String descending = "";
for (int i = charArray.length - 1; i >= 0; i--) {
    descending += charArray[i];
}

System.out.println("String in Descending order: " + descending);
}
}
10c

// Abstract ShoppingCart with Payment interface
import java.util.ArrayList;
import java.util.List;


// Interface for payment methods
interface Payment {
    void pay(double amount);
}

```

```
// Abstract class for a shopping cart

abstract class ShoppingCart {

    List<String> items = new ArrayList<>();

    double total = 0.0;


    // Each cart decides how checkout works

    abstract void checkout(Payment paymentMethod);


    void addItem(String item, double price) {

        items.add(item);

        total += price;

        System.out.println("Added: " + item + " (Price: " + price + ")");

    }


    void showCart() {

        System.out.println("Items in cart: " + items);

        System.out.println("Total amount: " + total);

    }

}


// Concrete cart for groceries

class GroceryCart extends ShoppingCart {

    @Override

    void checkout(Payment paymentMethod) {

        showCart();

        System.out.println("Proceeding to payment...");

        paymentMethod.pay(total);

    }

}


// Payment Method 1: Card
```

```
class CardPayment implements Payment {  
    private String cardNumber;  
  
    CardPayment(String cardNumber) {  
        this.cardNumber = cardNumber;  
    }  
  
    @Override  
    public void pay(double amount) {  
        System.out.println("Paid " + amount + " using card ending with "  
            + cardNumber.substring(cardNumber.length() - 4));  
    }  
}
```

// Payment Method 2: UPI

```
class UpiPayment implements Payment {  
    private String upid;  
  
    UpiPayment(String upid) {  
        this.upid = upid;  
    }  
  
    @Override  
    public void pay(double amount) {  
        System.out.println("Paid " + amount + " using UPI ID " + upid);  
    }  
}
```

// Main class

```
public class ShoppingDemo {  
    public static void main(String[] args) {
```



```
ShoppingCart cart = new GroceryCart();
```

```
cart.addItem("Milk", 40);
```

```
cart.addItem("Bread", 30);
```

```
Payment paymentMethod = new CardPayment("1234567812345678");
```

```
cart.checkout(paymentMethod);
```

```
Payment upi = new UpiPayment("user@upi");
```

```
cart.checkout(upi);
```

```
}
```

```
}
```